⚛️ REACT JS COMPLETE ROADMAP

Full Stack Web Development + Google Login + Real Projects

📋 PLAN OVERVIEW

Student:	Shanthosh S
Duration:	2 Weeks (React) + 1 Week (React Native)
Goal:	Job Ready in 1.5 Months
Resource:	W3Schools + Official React Docs
Daily Time:	4-5 hours

📅 DAILY SCHEDULE

Time	Activity
9:00 AM - 12:00 PM	Theory + W3Schools (3 hours)
12:00 PM - 1:00 PM	Lunch
1:00 PM - 3:00 PM	Code Practice (2 hours)
6:00 PM - 7:00 PM	Review + Problems

📅 WEEK 1: REACT BASICS

Day	Topics
Day 1	What is React + Setup + JSX + Components
Day 2	Props + useState Hook
Day 3	useEffect + API Calls
Day 4	React Router (Navigation)
Day 5	Forms + Validation
Day 6	Google Login (Firebase)
Day 7	Small Project + Review

🎯 DAY 1: WHAT IS REACT + JSX + COMPONENTS

📚 What is React?

- React is a JavaScript library for building user interfaces
- Created by Facebook/Meta in 2013
- Component-based (build small pieces, combine them)
- Uses Virtual DOM (fast updates)
- React = Web | React Native = Mobile (same concepts!)

📚 Setup React App

```
# Install Node.js first (nodejs.org)
# Then create React app:

npx create-react-app my-app
cd my-app
npm start

# OR use Vite (faster, recommended):
npm create vite@latest my-app -- --template react
```

```
cd my-app
npm install
npm run dev
```

📚 What is JSX?

```
// JSX = JavaScript + HTML mixed together!
// Looks like HTML but it's JavaScript!

function App() {
    const name = "Shanthosh";
    const age = 23;

    return (
        // JSX starts here
        <div>
            <h1>Hello {name}!</h1>
            <p>Age: {age}</p>
            <p>{age >= 18 ? "Adult" : "Minor"}</p>
        </div>
        // JSX ends here
    );
}

// JSX RULES:
// 1. Return ONE parent element
// 2. Use className instead of class
// 3. Use {} for JavaScript expressions
// 4. Self-closing tags: <img /> <input />
// 5. camelCase attributes: onClick, onChange
```

📚 Components

```
// Component = reusable piece of UI
// Like a function that returns JSX!

// FUNCTION COMPONENT (Modern way):
function UserCard() {
    return (
        <div className="card">
            <h2>Shanthosh S</h2>
            <p>React Developer</p>
        </div>
    );
```

```
}

// Arrow function component:
const UserCard = () => {
    return (
        <div>
            <h2>Shanthosh S</h2>
        </div>
    );
}

// USE component inside another component:
function App() {
    return (
        <div>
            <UserCard />  // Use like HTML tag!
            <UserCard />  // Reuse it!
            <UserCard />  // Again!
        </div>
    );
}
```

🧪 Brother's Test / Self Test:

1. "What is JSX?"
2. "What is a React component?"
3. "Create a simple component that shows your name"
4. "What's the difference between React and React Native?"

🎯 DAY 2: PROPS + useState

📚 Props (Like function parameters!)

```
// Props = data passed FROM parent TO child
// Like function arguments!

// PARENT:
function App() {
    return (
        <div>
            <UserCard name="Shanthosh" age={23} />
            <UserCard name="Kumar" age={25} />
        </div>
    );
}

// CHILD receives props:
function UserCard({ name, age }) {  // destructuring!
    return (
        <div>
            <h2>{name}</h2>
            <p>Age: {age}</p>
        </div>
    );
}

// RULES:
// Props are READ-ONLY (can't change them!)
// Pass strings: name="John"
// Pass numbers: age={23}
// Pass booleans: isActive={true}
// Pass arrays: items={[1,2,3]}
```

📚 useState Hook

```
import { useState } from 'react';

function Counter() {
    // [currentValue, functionToChange] = useState(initialValue)
    const [count, setCount] = useState(0);
    const [name, setName] = useState("Shanthosh");
```

```
    const [isVisible, setIsVisible] = useState(true);

    return (
        <div>
            <h1>Count: {count}</h1>
            <button onClick={() => setCount(count + 1)}>
                Add
            </button>
            <button onClick={() => setCount(count - 1)}>
                Minus
            </button>
            <button onClick={() => setCount(0)}>
                Reset
            </button>

            <input
                value={name}
                onChange={(e) => setName(e.target.value)}
            />
            <p>Hello {name}!</p>

            <button onClick={() => setIsVisible(!isVisible)}>
                Toggle
            </button>
            {isVisible && <p>I am visible!</p>}
        </div>
    );
}

// STATE WITH ARRAYS:
function TodoList() {
    const [todos, setTodos] = useState([]);
    const [input, setInput] = useState("");

    const addTodo = () => {
        setTodos([...todos, input]);  // spread + add new!
        setInput("");
    };

    return (
        <div>
            <input
                value={input}
                onChange={(e) => setInput(e.target.value)}
            />
            <button onClick={addTodo}>Add</button>
            <ul>
```

```
                {todos.map((todo, index) => (
                    <li key={index}>{todo}</li>
                ))}
            </ul>
        </div>
    );
}
```

🧪 Self Test:

1. "What are props? Can child change props?"
2. "What is useState? What does it return?"
3. "Build a counter with add, minus, reset buttons"
4. "Build a todo list with useState"

🎯 DAY 3: useEffect + API CALLS

📚 useEffect Hook

```
import { useState, useEffect } from 'react';

// useEffect = run code after render
// Used for: API calls, timers, subscriptions

function App() {
    const [data, setData] = useState([]);
    const [loading, setLoading] = useState(false);
    const [count, setCount] = useState(0);

    // 1. Run ONCE on mount (empty array [])
    useEffect(() => {
        console.log("Runs once when component loads!");
        fetchData();
    }, []);

    // 2. Run when count CHANGES
    useEffect(() => {
        console.log("Count changed to: " + count);
    }, [count]);

    // 3. Run on EVERY render (no array)
    useEffect(() => {
        console.log("Runs every render!");
    });

    // API Call inside useEffect:
    async function fetchData() {
        setLoading(true);
        try {
            const res = await fetch(
                'https://jsonplaceholder.typicode.com/posts'
            );
            const posts = await res.json();
            setData(posts);
        } catch(err) {
            console.log(err);
        } finally {
            setLoading(false);
```

```
        }
    }

    if(loading) return <p>Loading...</p>;

    return (
        <div>
            <h1>Posts ({data.length})</h1>
            {data.map(post => (
                <div key={post.id}>
                    <h3>{post.title}</h3>
                    <p>{post.body}</p>
                </div>
            ))}
        </div>
    );
}
```

📚 Conditional Rendering

```
// Show different UI based on condition:

function App() {
    const [isLoggedIn, setIsLoggedIn] = useState(false);
    const [loading, setLoading] = useState(true);

    // 1. If/Else
    if(loading) return <p>Loading...</p>;

    // 2. Ternary operator
    return (
        <div>
            {isLoggedIn ? (
                <h1>Welcome back!</h1>
            ) : (
                <h1>Please login!</h1>
            )}

            {/* 3. && operator (show if true) */}
            {isLoggedIn && <p>You are logged in!</p>}
        </div>
    );
}
```

🧪 Self Test:

1. "What are the 3 useEffect patterns?"
2. "Fetch posts from JSONPlaceholder and display"
3. "Show loading spinner while fetching"
4. "Show error message if fetch fails"

🎯 DAY 4: REACT ROUTER (NAVIGATION)

📚 Setup React Router

```
// Install:
npm install react-router-dom

// MAIN APP SETUP:
import { BrowserRouter, Routes, Route, Link, useNavigate } from 'react-router-dom

function App() {
    return (
        <BrowserRouter>
            {/* Navigation Links */}
            <nav>
                <Link to="/">Home</Link>
                <Link to="/about">About</Link>
                <Link to="/dashboard">Dashboard</Link>
            </nav>

            {/* Routes */}
            <Routes>
                <Route path="/" element={<Home />} />
                <Route path="/about" element={<About />} />
                <Route path="/dashboard" element={<Dashboard />} />
                <Route path="/user/:id" element={<UserDetail />} />
                <Route path="*" element={<NotFound />} />
            </Routes>
        </BrowserRouter>
    );
}
```

📚 Navigation + URL Params

```
import { useNavigate, useParams } from 'react-router-dom';

// Navigate programmatically:
function LoginPage() {
    const navigate = useNavigate();

    const handleLogin = () => {
        // After login success:
```

```
        navigate('/dashboard');
        // Or go back:
        navigate(-1);
    };

    return <button onClick={handleLogin}>Login</button>;
}

// Get URL parameters:
// URL: /user/123
function UserDetail() {
    const { id } = useParams();  // id = "123"

    return <h1>User ID: {id}</h1>;
}
```

📚 Protected Routes (Important!)

```
// Redirect if not logged in!

function ProtectedRoute({ children }) {
    const isLoggedIn = localStorage.getItem('isLoggedIn');

    if(!isLoggedIn) {
        return <Navigate to="/login" />;
    }

    return children;
}

// USE IN ROUTES:
<Routes>
    <Route path="/login" element={<Login />} />
    <Route
        path="/dashboard"
        element={
            <ProtectedRoute>
                <Dashboard />
            </ProtectedRoute>
        }
    />
</Routes>
```

🧪 Self Test:

1. "What is React Router used for?"
2. "Difference between Link and useNavigate?"
3. "How to get URL params?"
4. "Build protected route that redirects if not logged in"

🎯 DAY 5: FORMS + VALIDATION

📚 Controlled Forms

```
import { useState } from 'react';

function RegisterForm() {
    const [formData, setFormData] = useState({
        name: '',
        email: '',
        password: '',
        confirmPassword: ''
    });
    const [errors, setErrors] = useState({});

    // Handle all inputs with one function!
    const handleChange = (e) => {
        setFormData({
            ...formData,
            [e.target.name]: e.target.value
        });
    };

    // Validation
    const validate = () => {
        const newErrors = {};

        if(!formData.name) {
            newErrors.name = "Name is required!";
        }

        if(!formData.email.includes('@')) {
            newErrors.email = "Invalid email!";
        }

        if(formData.password.length < 6) {
            newErrors.password = "Min 6 characters!";
        }

        if(formData.password !== formData.confirmPassword) {
            newErrors.confirmPassword = "Passwords don't match!";
        }
```

```
        return newErrors;
    };

    const handleSubmit = (e) => {
        e.preventDefault();  // Stop page reload!

        const validationErrors = validate();

        if(Object.keys(validationErrors).length > 0) {
            setErrors(validationErrors);
            return;
        }

        // All good! Submit data
        console.log("Form submitted:", formData);
        // Call API here
    };

    return (
        <form onSubmit={handleSubmit}>
            <div>
                <input
                    name="name"
                    value={formData.name}
                    onChange={handleChange}
                    placeholder="Full Name"
                />
                {errors.name && <p style={{color:'red'}}>{errors.name}</p>}
            </div>

            <div>
                <input
                    name="email"
                    value={formData.email}
                    onChange={handleChange}
                    placeholder="Email"
                />
                {errors.email && <p style={{color:'red'}}>{errors.email}</p>}
            </div>

            <div>
                <input
                    type="password"
                    name="password"
                    value={formData.password}
                    onChange={handleChange}
                    placeholder="Password"
```

```
                />
                {errors.password && <p style={{color:'red'}}>{errors.password}</p
            </div>

            <div>
                <input
                    type="password"
                    name="confirmPassword"
                    value={formData.confirmPassword}
                    onChange={handleChange}
                    placeholder="Confirm Password"
                />
                {errors.confirmPassword && (
                    <p style={{color:'red'}}>{errors.confirmPassword}</p>
                )}
            </div>

            <button type="submit">Register</button>
        </form>
    );
}
```

🎯 DAY 6: GOOGLE LOGIN (FIREBASE)

⚠️ **IMPORTANT:** Google Login uses Firebase Auth - FREE and easy to set up!

📚 Step 1: Firebase Setup

```
// 1. Go to console.firebase.google.com
// 2. Create new project
// 3. Add Web app
// 4. Enable Authentication → Google Sign In
// 5. Copy your config

// Install Firebase:
npm install firebase
```

📚 Step 2: Firebase Config File

```
// src/firebase.js
import { initializeApp } from 'firebase/app';
import { getAuth, GoogleAuthProvider } from 'firebase/auth';

const firebaseConfig = {
    apiKey: "YOUR_API_KEY",
    authDomain: "YOUR_PROJECT.firebaseapp.com",
    projectId: "YOUR_PROJECT_ID",
    storageBucket: "YOUR_PROJECT.appspot.com",
    messagingSenderId: "YOUR_SENDER_ID",
    appId: "YOUR_APP_ID"
};

const app = initializeApp(firebaseConfig);
export const auth = getAuth(app);
export const googleProvider = new GoogleAuthProvider();
```

📚 Step 3: Google Login Component

```
import { auth, googleProvider } from './firebase';
import { signInWithPopup, signOut } from 'firebase/auth';
import { useState } from 'react';
```

```
function GoogleLogin() {
    const [user, setUser] = useState(null);
    const [loading, setLoading] = useState(false);

    // GOOGLE LOGIN
    const handleGoogleLogin = async () => {
        setLoading(true);
        try {
            const result = await signInWithPopup(auth, googleProvider);
            const loggedUser = result.user;

            setUser({
                name: loggedUser.displayName,
                email: loggedUser.email,
                photo: loggedUser.photoURL,
                uid: loggedUser.uid
            });

            console.log("Logged in:", loggedUser.displayName);

        } catch(error) {
            console.log("Login failed:", error.message);
        } finally {
            setLoading(false);
        }
    };

    // LOGOUT
    const handleLogout = async () => {
        try {
            await signOut(auth);
            setUser(null);
            console.log("Logged out!");
        } catch(error) {
            console.log("Logout failed:", error.message);
        }
    };

    // NOT LOGGED IN
    if(!user) {
        return (
            <div>
                <h2>Please Login</h2>
                <button
                    onClick={handleGoogleLogin}
                    disabled={loading}
                    style={{
```

```
                        background: '#4285f4',
                        color: 'white',
                        padding: '10px 20px',
                        border: 'none',
                        borderRadius: '5px',
                        cursor: 'pointer'
                    }}
                >
                    {loading ? 'Loading...' : '🔵 Sign in with Google'}
                </button>
            </div>
        );
    }

    // LOGGED IN
    return (
        <div>
            <img src={user.photo} alt="profile" width="50" />
            <h2>Welcome, {user.name}!</h2>
            <p>{user.email}</p>
            <button onClick={handleLogout}>Logout</button>
        </div>
    );
}
```

📚 Step 4: Keep User Logged In

```
import { onAuthStateChanged } from 'firebase/auth';

function App() {
    const [user, setUser] = useState(null);

    // Check if user is already logged in
    useEffect(() => {
        const unsubscribe = onAuthStateChanged(auth, (currentUser) => {
            if(currentUser) {
                setUser(currentUser);
            } else {
                setUser(null);
            }
        });

        // Cleanup
        return () => unsubscribe();
    }, []);
```

```
    return (
        <div>
            {user ? <Dashboard user={user} /> : <LoginPage />}
        </div>
    );
}
```

🎯 DAY 7: SMALL PROJECT - LOGIN + DASHBOARD

📚 Build This Project:

```
PROJECT STRUCTURE:
src/
  ├── App.jsx
  ├── firebase.js
  ├── pages/
  │    ├── Login.jsx
  │    ├── Register.jsx
  │    └── Dashboard.jsx
  ├── components/
  │    ├── Navbar.jsx
  │    └── ProtectedRoute.jsx
  └── index.css

FEATURES:
✅ Register (email + password)
✅ Login (email + password)
✅ Google Login (Firebase)
✅ Dashboard (protected route)
✅ Logout button
✅ Show user name + email
✅ Fetch posts from API
✅ Display in list
```

📅 WEEK 2: REACT INTERMEDIATE

Day	Topics
Day 1	Context API + useContext (Global State)
Day 2	Custom Hooks
Day 3	useCallback + useMemo (Performance)
Day 4	File Upload + Image Preview
Day 5	Connect to Node.js Backend
Day 6-7	Full Stack Project

🎯 WEEK 2 - DAY 1: CONTEXT API

📚 What is Context API?

```
// Problem: Prop drilling is messy!
// Solution: Context API - share data everywhere!

// 1. CREATE CONTEXT:
import { createContext, useContext, useState } from 'react';

const AuthContext = createContext();

// 2. CREATE PROVIDER:
export function AuthProvider({ children }) {
    const [user, setUser] = useState(null);
    const [isLoggedIn, setIsLoggedIn] = useState(false);

    const login = (userData) => {
        setUser(userData);
        setIsLoggedIn(true);
        localStorage.setItem('user', JSON.stringify(userData));
    };

    const logout = () => {
        setUser(null);
        setIsLoggedIn(false);
```

```
        localStorage.removeItem('user');
    };

    return (
        <AuthContext.Provider value={{ user, isLoggedIn, login, logout }}>
            {children}
        </AuthContext.Provider>
    );
}

// 3. WRAP YOUR APP:
// index.js
<AuthProvider>
    <App />
</AuthProvider>

// 4. USE IN ANY COMPONENT:
function Navbar() {
    const { user, isLoggedIn, logout } = useContext(AuthContext);

    return (
        <nav>
            {isLoggedIn ? (
                <>
                    <span>Hello {user.name}!</span>
                    <button onClick={logout}>Logout</button>
                </>
            ) : (
                <Link to="/login">Login</Link>
            )}
        </nav>
    );
}
```

🎯 WEEK 2 - DAY 5: CONNECT TO NODE BACKEND

📚 Full Stack Connection

```
// YOUR NODE BACKEND (Express):
// app.js
const express = require('express');
```

```
const cors = require('cors');
const app = express();

app.use(cors());  // Allow React to connect!
app.use(express.json());

app.get('/api/posts', (req, res) => {
    res.json([
        { id: 1, title: "Post 1" },
        { id: 2, title: "Post 2" }
    ]);
});

app.listen(5000);

// REACT FRONTEND:
async function getPosts() {
    try {
        const res = await fetch('http://localhost:5000/api/posts');
        const posts = await res.json();
        setPosts(posts);
    } catch(err) {
        console.log(err);
    }
}

// JWT AUTH WITH BACKEND:
async function login(email, password) {
    try {
        const res = await fetch('http://localhost:5000/api/login', {
            method: 'POST',
            headers: { 'Content-Type': 'application/json' },
            body: JSON.stringify({ email, password })
        });
        const data = await res.json();

        // Save JWT token
        localStorage.setItem('token', data.token);

    } catch(err) {
        console.log(err);
    }
}

// Send token with requests:
async function getProtectedData() {
    const token = localStorage.getItem('token');
```

```
    const res = await fetch('http://localhost:5000/api/protected', {
        headers: {
            'Authorization': `Bearer ${token}`
        }
    });
    const data = await res.json();
    return data;
}
```

🌐 IMPORTANT WEB THINGS TO KNOW

1. CORS

```
// Cross-Origin Resource Sharing
// React (port 3000) calling Node (port 5000) = CORS error!
// Fix in Node.js:
npm install cors

const cors = require('cors');
app.use(cors());

// OR specific origin:
app.use(cors({ origin: 'http://localhost:3000' }));
```

2. Environment Variables

```
// .env file (never commit to GitHub!)
REACT_APP_API_URL=http://localhost:5000
REACT_APP_FIREBASE_API_KEY=your_key

// USE IN REACT:
const apiUrl = process.env.REACT_APP_API_URL;

// Vite uses VITE_ prefix:
VITE_API_URL=http://localhost:5000
const apiUrl = import.meta.env.VITE_API_URL;
```

3. localStorage in React

```
// Store user data:
localStorage.setItem('user', JSON.stringify(user));
localStorage.setItem('token', token);

// Get user data:
const user = JSON.parse(localStorage.getItem('user'));
const token = localStorage.getItem('token');

// Remove:
localStorage.removeItem('user');
localStorage.clear();
```

4. JWT Token Auth Flow

```
// FLOW:
// 1. User logs in → Backend returns JWT token
// 2. React saves token in localStorage
// 3. Every request sends token in header
// 4. Backend verifies token

// Frontend:
const token = localStorage.getItem('token');

fetch('/api/data', {
    headers: {
        'Authorization': `Bearer ${token}`
    }
});

// Backend (Node.js):
const jwt = require('jsonwebtoken');

// Create token:
const token = jwt.sign(
    { userId: user.id },
    'your_secret_key',
    { expiresIn: '7d' }
);

// Verify token (middleware):
const verified = jwt.verify(token, 'your_secret_key');
```

5. Deploy React App

```
// VERCEL (Easiest!):
npm install -g vercel
vercel

// NETLIFY:
npm run build
// Drag build folder to netlify.com

// GITHUB PAGES:
npm install gh-pages
// Add to package.json:
"homepage": "https://username.github.io/repo"
"predeploy": "npm run build"
```

```
"deploy": "gh-pages -d build"
npm run deploy
```

📱 WEEK 3: REACT NATIVE (PROPERLY)

Day	Topics
Day 1	Setup + Core Components (View, Text, TouchableOpacity)
Day 2	Stack + Tab Navigation properly
Day 3	FlatList + API calls + Loading states
Day 4	AsyncStorage + Forms + Validation
Day 5	Google Login (Firebase - same as React!)
Day 6-7	Polish Attendance App + Build new app

📚 React vs React Native Comparison:

React (Web)	React Native (Mobile)
	<View>
<p>, <h1>	<Text>
<img>	<Image>
<button>	<TouchableOpacity>
<input>	<TextInput>
	<FlatList>
CSS	StyleSheet
onClick	onPress
localStorage	AsyncStorage

✅ COMPLETE CHECKLIST

Week 1 - React Basics:

- ☐ JSX and Components
- ☐ Props
- ☐ useState
- ☐ useEffect + API calls
- ☐ React Router
- ☐ Protected Routes
- ☐ Forms + Validation
- ☐ Google Login (Firebase)
- ☐ Small Project

Week 2 - React Intermediate:

- ☐ Context API (Global State)
- ☐ Custom Hooks
- ☐ File Upload
- ☐ Connect to Node Backend
- ☐ JWT Authentication
- ☐ Full Stack Project

Week 3 - React Native:

- ☐ Core Components
- ☐ Navigation properly
- ☐ AsyncStorage
- ☐ Google Login
- ☐ Polish existing apps
- ☐ New mobile project

Web Essentials:

- ☐ CORS handling
- ☐ Environment variables
- ☐ JWT Auth flow
- ☐ Deploy on Vercel
- ☐ Google Login (Firebase)

📚 Resources

Resource	URL
W3Schools React	w3schools.com/react
React Official Docs	react.dev
Firebase Console	console.firebase.google.com
React Router Docs	reactrouter.com
JSONPlaceholder API	jsonplaceholder.typicode.com
Deploy (Vercel)	vercel.com

🎯 GOAL: JOB IN 1.5 MONTHS!

JS ✅ → React ⚛️ → React Native 📱 → Projects 🚀 → JOB 💼

🔥 YOU'VE GOT THIS! 💪